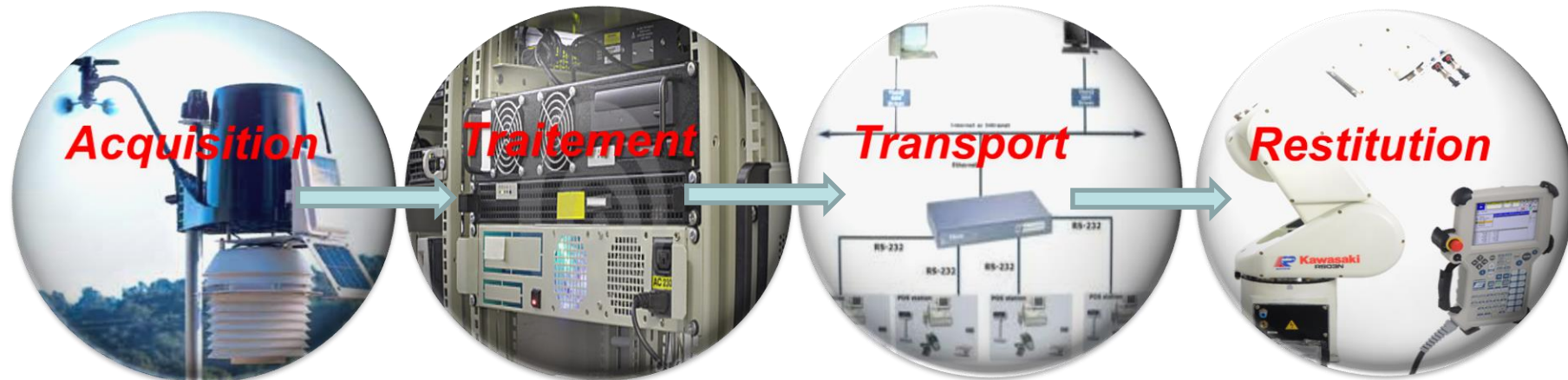


Des microcontrôleurs aux objets connectés

✚ Objectif de l'UE : Découvrir la programmation des systèmes embarqués à travers deux familles de microcontrôleurs :

- ✓ Partie 1 : Microcontrôleur ATMEGA 328
 - ✓ Les fondamentaux de la PPE
- ✓ Partie 2 : Microcontrôleur ESP32
 - ✓ Objets communicants et Internet des objets (IoT)





Des microcontrôleurs aux objets connectés

✚ Objectif de l'UE : Découvrir la programmation des systèmes embarqués à travers deux familles de microcontrôleurs :

- ✓ Partie 1 : Microcontrôleur ATMEGA 328
 - ✓ Les fondamentaux de la PPE
- ✓ Partie 2 : Microcontrôleur ESP32
 - ✓ Objets communicants et Internet des objets (IoT)

✚ Organisation des enseignements en 2 parties

- ✓ Formation :
 - ✓ CM : 3x1h30
 - ✓ TP : 3x 3h
- ✓ Evaluations.
 - ✓ 1 épreuve écrite : 1h
 - ✓ 1 épreuve pratique : 1h

Rappel:

CC1 le 12/11/26 14:30-16h

CC2 du 12/11/26 au 19/11/26



Partie I

TP 1

Programmation des Microcontrôleurs



Plan du Cours PPE

Des microcontrôleurs aux objets connectés

Objectif de l'UE : Découvrir la programmation des systèmes embarqués à travers deux familles de microcontrôleurs :

■ **Partie 1 — Les fondamentaux de l'embarqué**

- ✓ **Microcontrôleur ATmega 328 (Arduino)**
 - ✓ Architecture d'un système embarqué
 - ✓ Programmation C/C++ avec Arduino IDE
 - ✓ Gestion des entrées/sorties numériques et analogiques
 - ✓ Communication avec des capteurs et actionneurs
 - ✓ Mise en œuvre de petites applications embarquées



Plan du Cours PPE

Des microcontrôleurs aux objets connectés

Objectif de l'UE : Découvrir la programmation des systèmes embarqués à travers deux familles de microcontrôleurs :

■ **Partie 1 — Les fondamentaux de l'embarqué**

- ✓ **Applications étudiées :**
 - ✓ E/S TOR et Boutons et LEDs etc...
 - ✓ E/S analogique Potentiomètre, Capteurs (CAN), LEDs moteurs etc...
 - ✓ **Gestion bibliothèque (POO)**
 - ✓ Afficheur LCD RGB
 - ✓ Capteurs température / humidité
- ✓ **Gestion des interruptions:**
 - ✓ Boutons
 - ✓ Encodeur incrémental



Plan du Cours PPE

■ Partie 2 — Les objets communicants

✓ Microcontrôleur ESP32 :

- ✓ Architecture et ressources d'un SoC connecté
- ✓ Communication Wi-Fi
- ✓ Échanges de données IoT
- ✓ Protocoles TCP, webSocket et MQTT
- ✓ Au format JSON
- ✓ Supervision avec Node-RED
- ✓ Mise en œuvre d'objets connectés

✓ Applications étudiées :

- ✓ Acquisition de données capteurs
- ✓ Transmission réseau
- ✓ Commande distante
- ✓ Internet des objets (IoT)



Progression pédagogique

Une progression en 2 étapes : du matériel embarqué vers l'IoT

Étape 1 — Découvrir le monde embarqué

Support : Arduino / ATmega328

🎯 **Objectifs de la formation : comprendre comment programmer un microcontrôleur**

Séquence

Prise en main de l'environnement

Entrées / sorties numériques

Conversion analogique

Interfaces utilisateurs

Capteurs et actionneurs

Compétences développées

Arduino IDE, compilation, téléversement

Lire une information, commander un actionneur

Acquisition de grandeurs physiques

Affichage, interaction homme-machine

Réaliser une application embarquée complète

Évaluation des connaissances :

➡ ☐ QCM et questions de compréhension

Évaluation des compétences :

➡ ☐ Réalisation d'une application embarquée mettant en œuvre des capteurs et/ou des actionneurs



Progression pédagogique

Une progression en 2 étapes : du matériel embarqué vers l'IoT

Étape 2 — Connecter le système embarqué

Support : ESP32

🎯 **Objectifs de la formation : transformer un système embarqué en objet connecté**

Séquence

Découverte ESP32

Communication réseau

Protocoles TCP, websocket et MQTT

Serveur IoT

Application connectée

Compétences développées

Ressources matérielles, GPIO, Wi-Fi

Connexion à un réseau IP

Publication / abonnement de données

Broker MQTT, Node-RED

Supervision et échange de données

Évaluation des connaissances :

➡ ☐ QCM et questions de compréhension

Évaluation des compétences :

➡ ☐ Réalisation d'un objet connecté communicant

Programmation Pour l'Embarqué

Les systèmes embarqués sont partout :
Où trouve-t-on des systèmes embarqués ?



🚗 Automobile :

Plus de 100 calculateurs électroniques dans un véhicule moderne

🏠 Domotique / IoT :

Régulation intelligente du chauffage, système de surveillance (caméras, alarme)



🏭 Robot industriel :

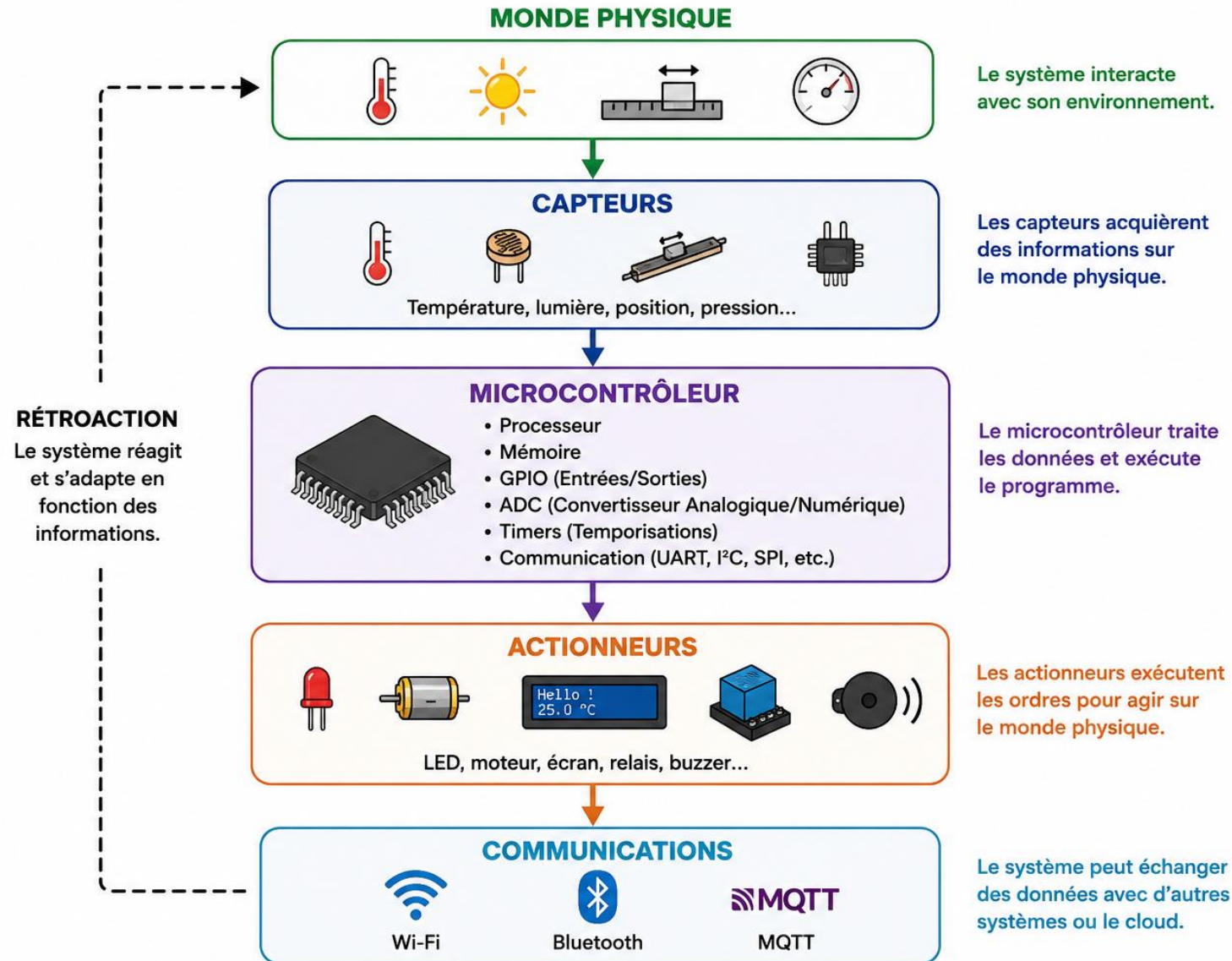
Contrôle-commande en temps réel

🕒 Montre connectée :

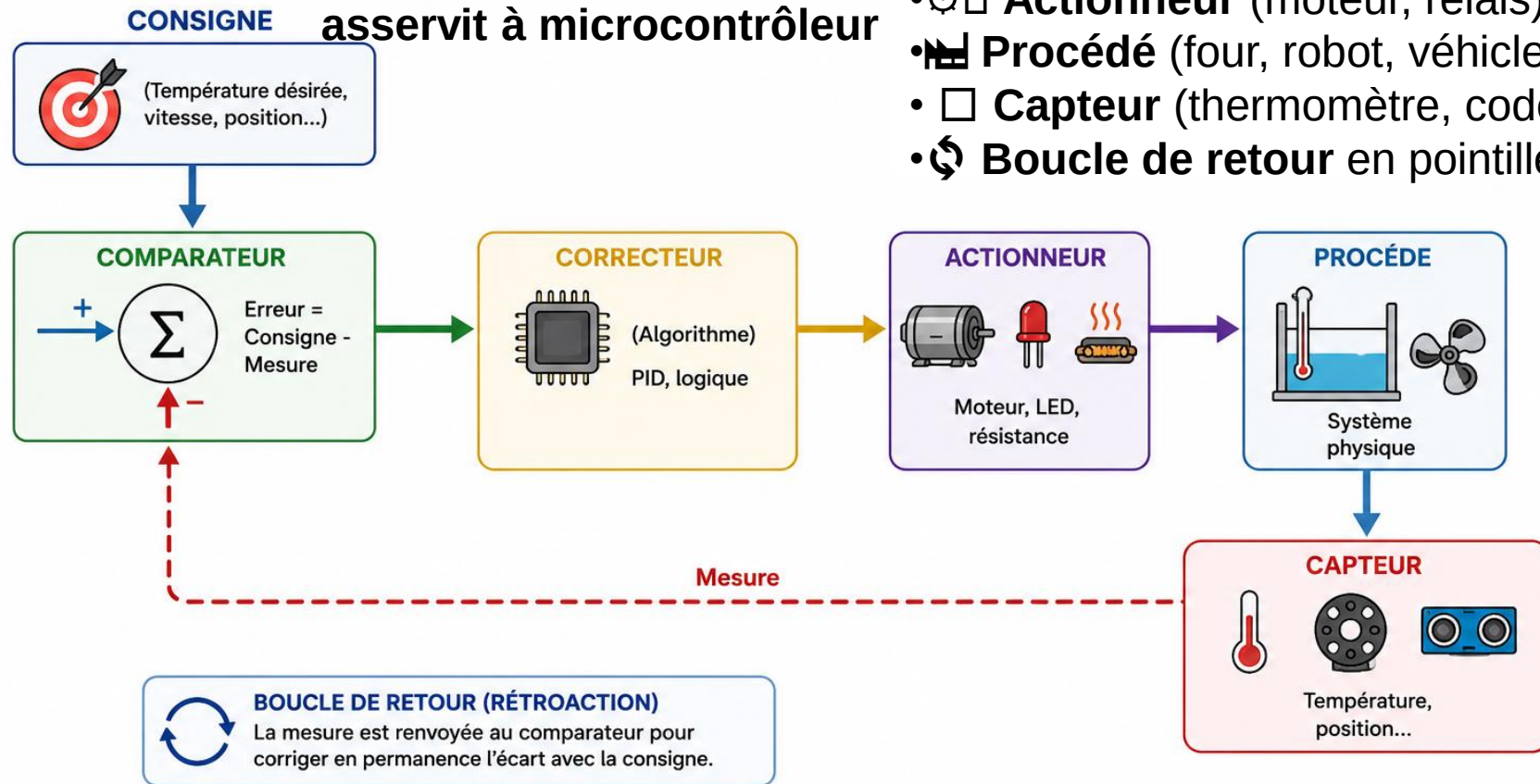
Acquisition et traitement de données physiologiques



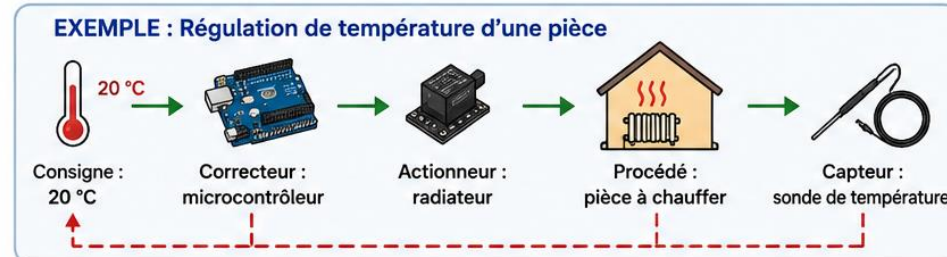
Architecture du système microcontrôleur



Architecture d'un système asservit à microcontrôleur



- **Consigne** (icône cible ou potentiomètre)
- **Comparateur** (balance ou symbole Σ)
- **Correcteur / Microcontrôleur** (puce électronique)
- **Actionneur** (moteur, relais)
- **Procédé** (four, robot, véhicule...)
- **Capteur** (thermomètre, codeur, capteur de distance)
- **Boucle de retour** en pointillés colorés



- Consigne : valeur de référence
- Erreur : écart entre consigne et mesure
- Calcul et décision du correcteur
- Commande de l'actionneur
- Action sur le procédé
- Mesure du résultat
- - - Retour d'information (rétroaction)



Applications des systèmes embarqués en asservissement

Qu'est-ce qu'un asservissement ?

Un asservissement est un système de commande automatique qui ajuste en permanence une action afin que la grandeur mesurée soit la plus proche possible de la valeur de consigne.

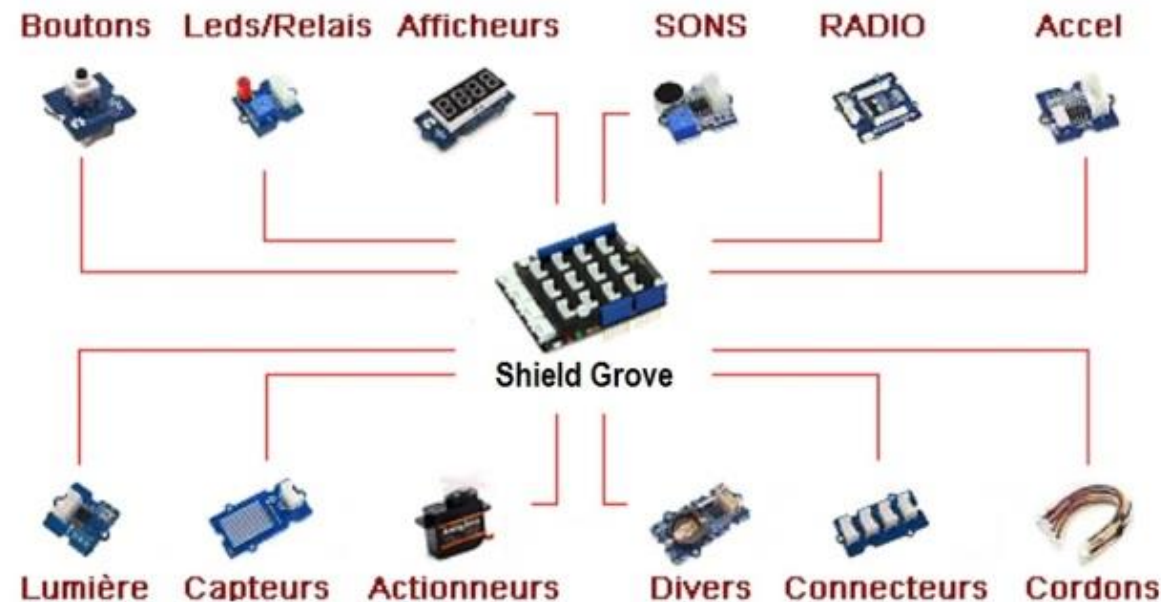
Système asservi	Grandeur asservie	Capteur	Actionneur
 Régulateur de vitesse automobile	Vitesse	Capteur ABS / vitesse	Papillon, moteur
☐ Thermostat connecté	Température	Sonde de température	Chauffage / climatisation
 Drone	Position / altitude	IMU, GPS, baromètre	Moteurs
☐ Robot industriel	Position	Codeurs	Servomoteurs
Ascenseur	Position des cabines	Capteurs de position	Moteur électrique
 Machine CNC	Position des axes	Codeurs	Moteurs pas à pas / servomoteurs

TP1 – Prise en main de l'environnement de développement Arduino

Objectifs du TP

À l'issue de cette séance, l'étudiant sera capable de :

- ⚙️ ☐ Installer et configurer l'IDE Arduino
- ☐ Découvrir l'architecture d'une carte Arduino Uno
- 🔍 Identifier les ressources du microcontrôleur ATmega328P
- 🔌 Configurer et exploiter les entrées/sorties GPIO (Digital I/O, entrées analogiques, PWM)
- 💻 Écrire, compiler et téléverser une application embarquée
- ☐ Utiliser les principales fonctions de l'API Arduino
- 📖 Exploiter les bibliothèques logicielles (Libraries)
- 🔍 Tester et déboguer une application avec les outils de développement Arduino





TP1 – Prise en main de l'environnement de développement Arduino

Liste des périphériques utilisés

boutons



Entrées TOR

leds



sorties TOR/analogique

potentiomètre



entrée analogique

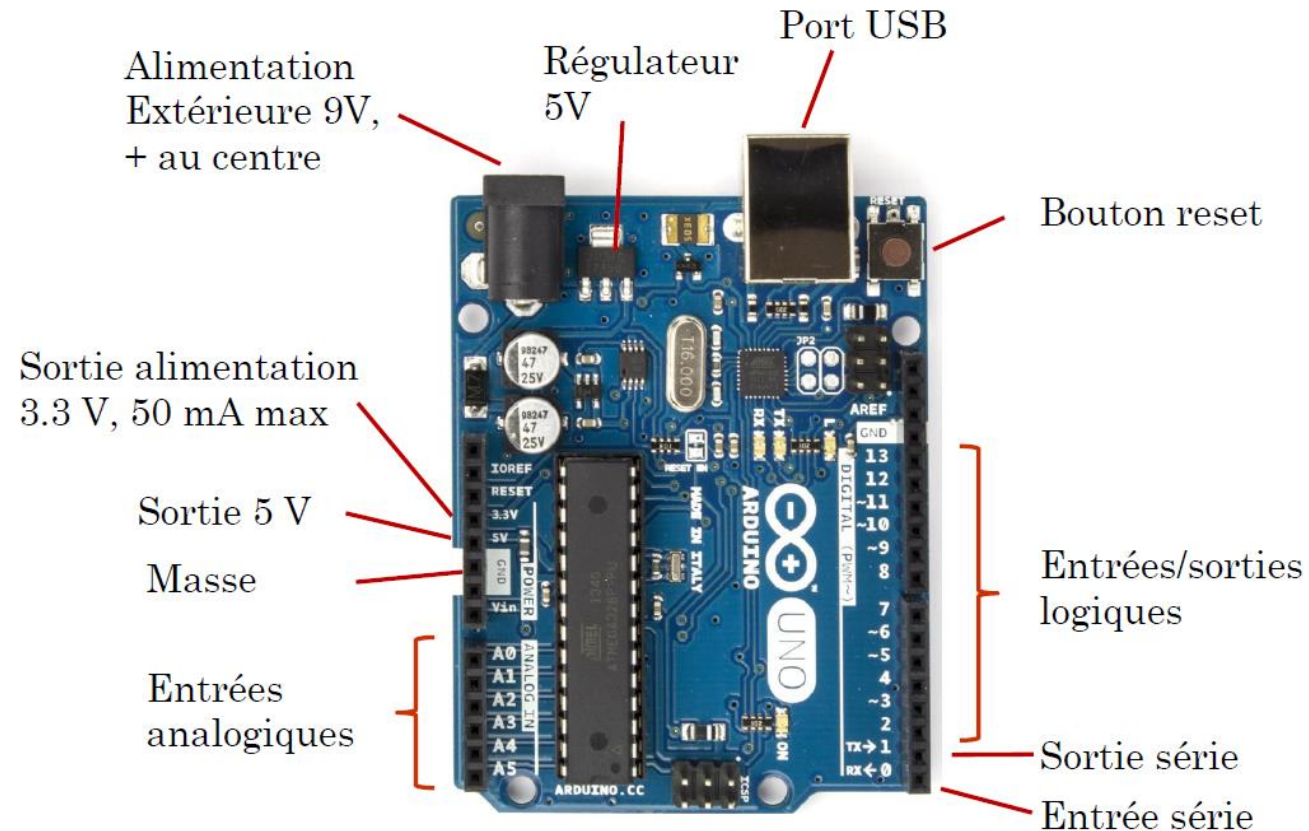


Le Microcontrôleur ATMEGA 328 sur carte Arduino Uno



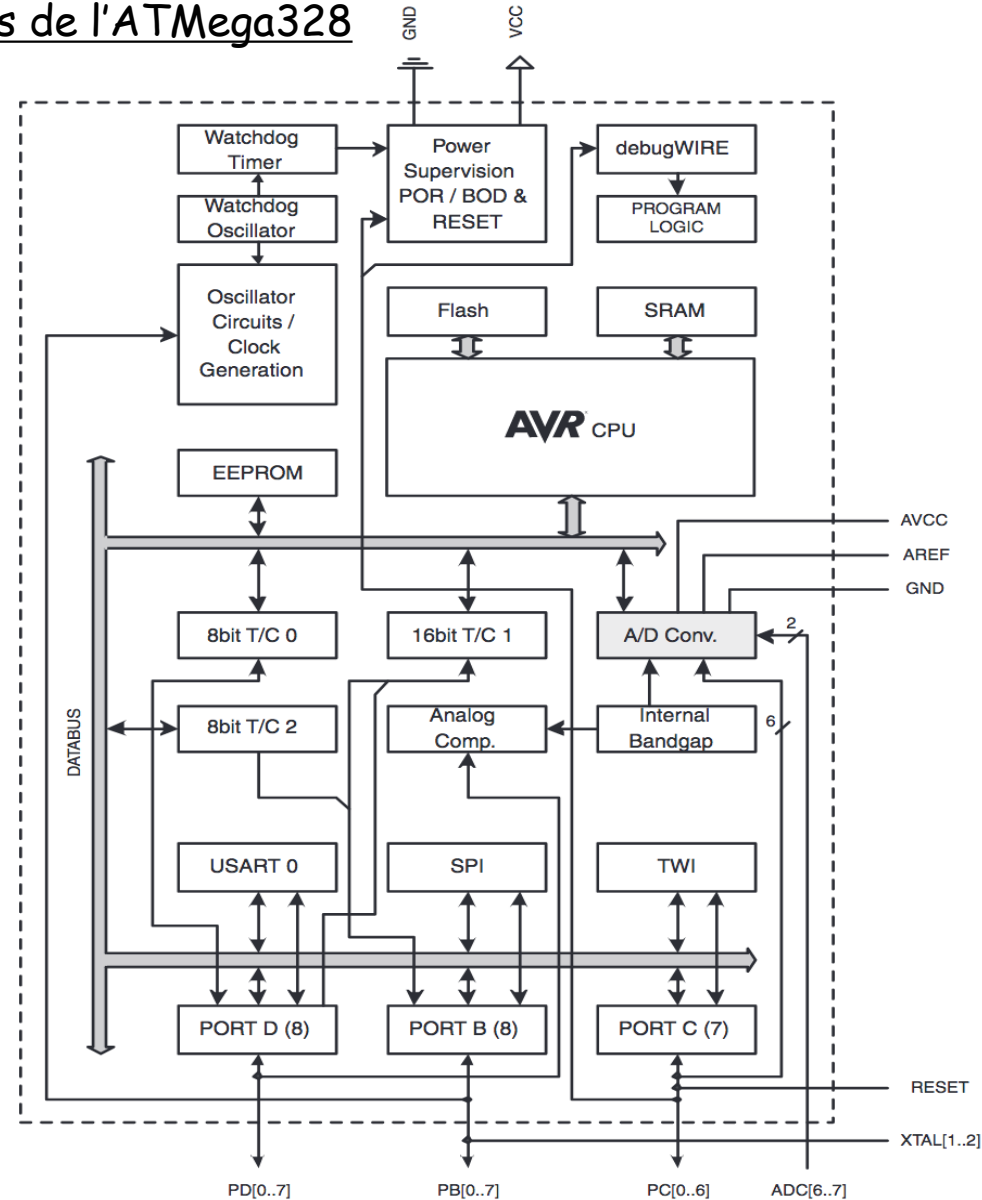
Programmation Pour l'Embarqué

La carte Arduino UNO est construite autour d'un microcontrôleur ATmega 328P cadencé à 16MHz.



Les microcontrôleurs pour l'embarqué représentent 96 à 98% des ventes contre 2% à 4% pour les processeurs (et microprocesseurs) de PC ou compatibles.

Ressources internes de l'ATMega328





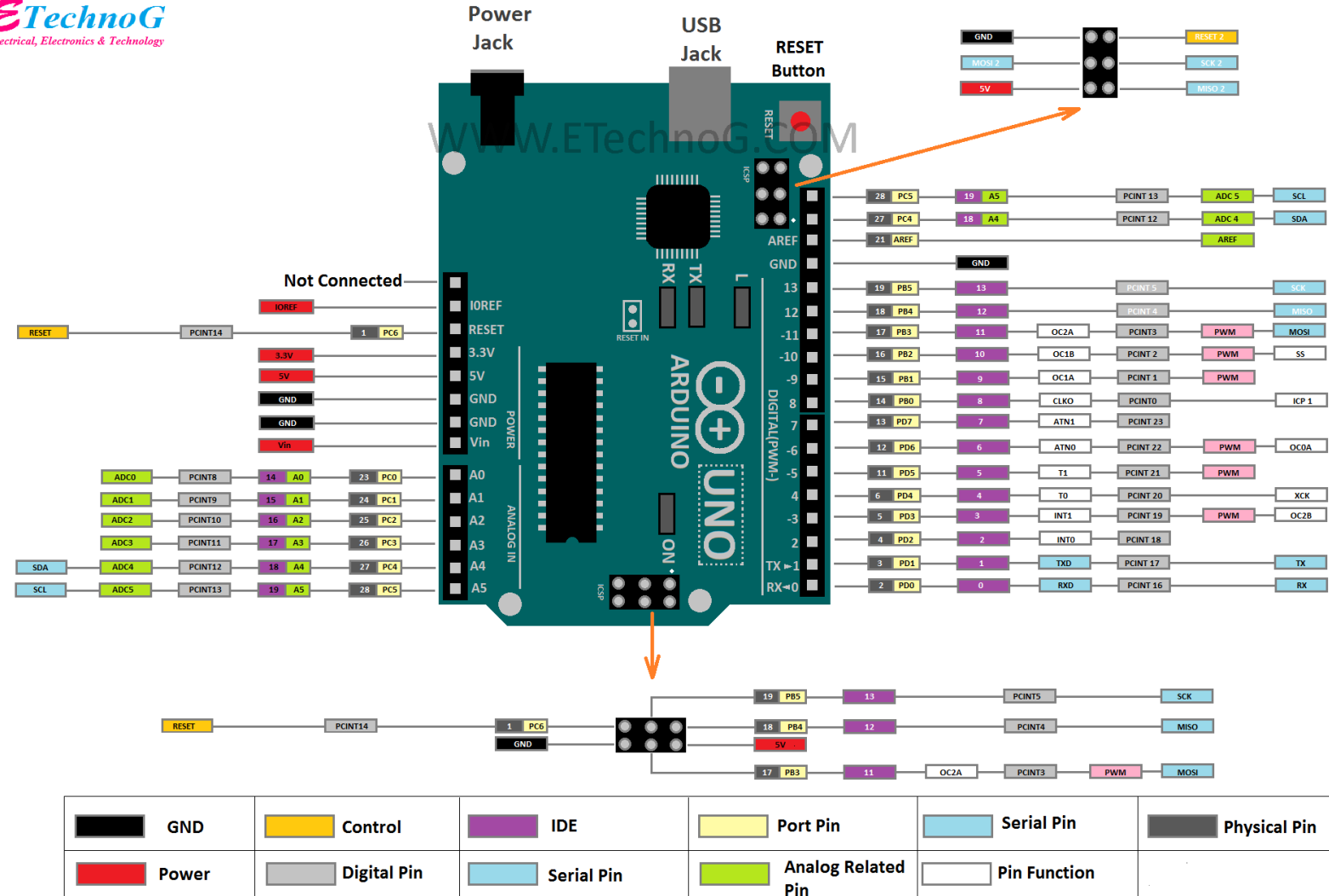
Ports d'EIS de l'ATMega328



CERI

Programmation Pour l'Embarqué

ETechnoG
Electrical, Electronics & Technology



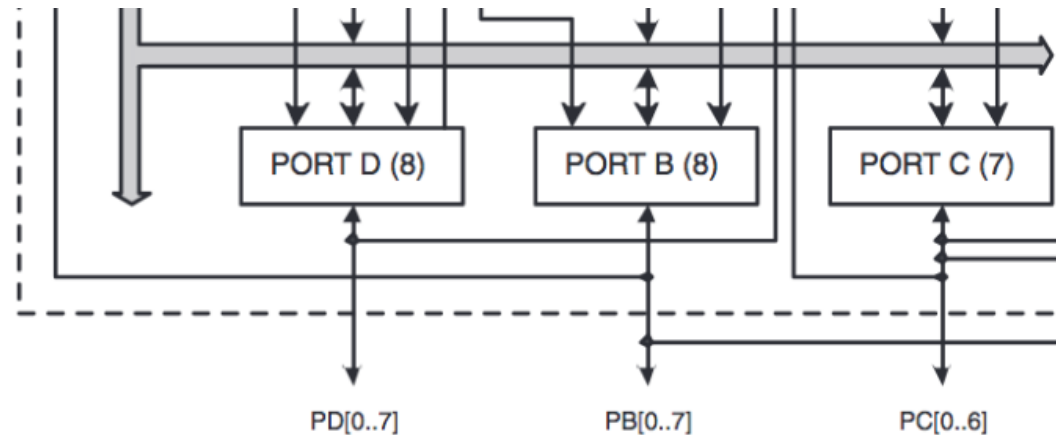
Arduino UNO Pinout Diagram

L2 Informatique

UE S-E06-0207



Ces ports ou périphériques d'entrées-sorties (GPIO) sont 3 ports digitaux (numériques) nommés **PORTB**, **PORTC** et **PORTD**. Les ports **B** et **D** sont complets c'est à dire qu'ils possèdent 8 bits. Le port **C** est incomplet puisqu'il n'en possède que 7 !



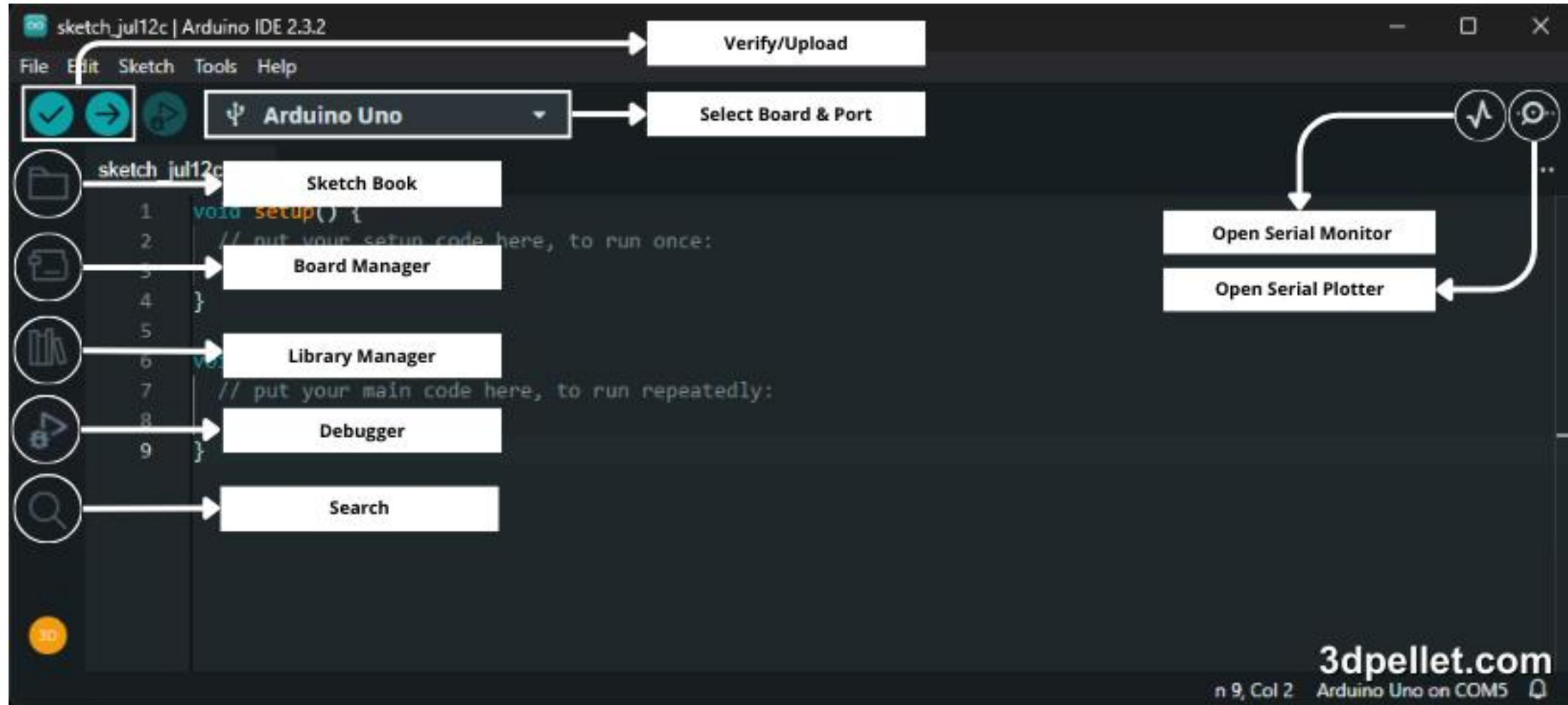
Note importante : Toutes les broches "GPIO" de l'ATMega328 sont « multiplexées » (Alternate Port functions) en ce sens qu'une broche donnée peut avoir plusieurs fonctionnalités. Ainsi par exemple la broche N°2 de l'Arduino Uno peut être configurée comme une E/S digitale, mais également comme l'entrée de réception de données de l'UART et enfin comme une entrée d'interruption sensible au changement d'état de cette dernière... **A suivre...**



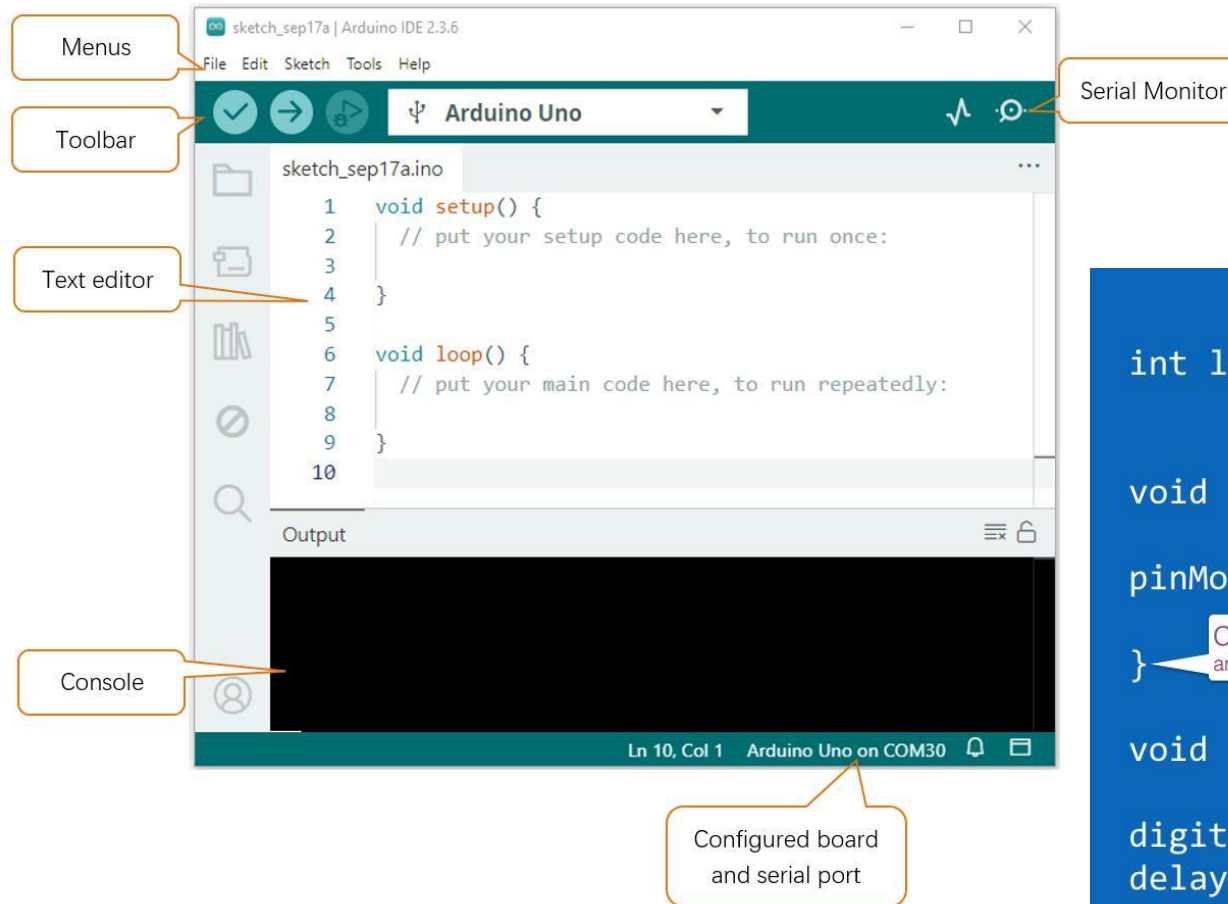
Présentation de l'IDE Arduino



IDE Arduino 2,X (utiliser la dernière version)



IDE Arduino 2,X (utiliser la dernière version)



```
int ledPin = 8;
```

Declaring - Variables, Integers, Constants, libraries

```
void setup(){
```

The setup() function is called when a sketch starts. Use it to initialize variables, pin modes, start using libraries, etc. The setup function will only run once, after each powerup or reset of the Arduino board.

```
pinMode (ledPin, OUTPUT);
```

```
}
```

Curly Braces - The curly braces {} indicate the start and the finish of functions and statements blocks.

Functions - are like verbs. They state the action. Functions usually contain parenthesis and a semicolon.

```
void loop (){
```

void loop function - repeats functions forever.

```
digitalWrite(ledPin, HIGH);
```

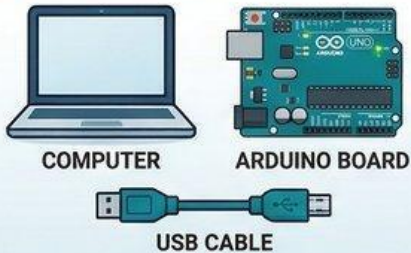
```
delay (1000);
```

```
digitalWrite(ledPin, LOW);
```

```
delay(1000);
```

Delay - Pauses program. Time interval in milliseconds. 1000th of a second = 1 second.

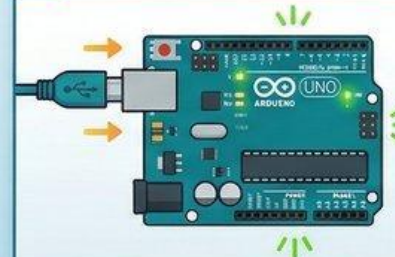
```
}
```


Les étapes de programmation**STEP-BY-STEP: PROGRAMMING YOUR ARDUINO FOR THE FIRST TIME****1. PREREQUISITES**

1. GATHER MATERIALS:
 Arduino board (Uno),
 USB data cable, Computer.

2. INSTALL IDE

2. INSTALL ARDUINO SOFTWARE (IDE):
 Download from official website (arduino.cc) & install.

3. CONNECT BOARD

3. CONNECT THE ARDUINO:
 Use the USB cable to connect board to computer.

4. SELECT BOARD & PORT

4. CONFIGURE IDE:
 Go to Tools > Board to select yours, then select the COM Port.

4. SELECT BOARD & PORT

4. OPEN BLINK SKETCH:
 Go to Tools > Board to select yours, select the COM Port.

5. OPEN 'BLINK' SKETCH

5. OPEN 'BLINK' EXAMPLE:
 File > Examples > Basics > Blink.

6. VERIFY & UPLOAD

6. UPLOAD THE CODE:
 First Verify (Checkmark) then Upload (Arrow). Watch the LEDs flicker.

7. SUCCESS & MODIFY

7. SUCCESS! LED BLINKS.
TRY MODIFYING DELAY!
 Change delay times to observe changes.



Présentation de l'API Arduino

L2 Informatique

UE S-E06-0207

L'API Arduino

API / fonction	Rôle	Syntaxe générique	Exemple
<code>pinMode()</code>	Configurer le mode d'une broche	<code>pinMode(broche, mode);</code>	<code>pinMode(35, INPUT);</code>
<code>digitalRead()</code>	Lire l'état d'une entrée numérique	<code>digitalRead(broche);</code>	<code>etat = digitalRead(26);</code>
<code>digitalWrite()</code>	Écrire un état logique sur une sortie	<code>digitalWrite(broche, valeur);</code>	<code>digitalWrite(14, HIGH);</code>
<code>analogRead()</code>	Lire une entrée analogique	<code>analogRead(broche);</code>	<code>valeur = analogRead(35);</code>
<code>analogWrite()</code>	Générer une sortie PWM*	<code>analogWrite(broche, valeur);</code>	<code>analogWrite(14, 128);</code>
<code>analogWriteResolution()</code>	Définir la résolution de la sortie PWM*	<code>analogWriteResolution(bits);</code>	<code>analogWriteResolution(8);</code>
<code>analogWriteFrequency()</code>	Définir la fréquence PWM*	<code>analogWriteFrequency(frequence);</code>	<code>analogWriteFrequency(5000);</code>
<code>delay()</code>	Créer une temporisation bloquante	<code>delay(duree);</code>	<code>delay(1000);</code>
<code>delayMicroseconds()</code>	Créer une temporisation en microsecondes	<code>delayMicroseconds(duree);</code>	<code>delayMicroseconds(10);</code>

L'API Arduino

API / fonction	Rôle	Syntaxe générique	Exemple
millis()	Obtenir le temps écoulé depuis le démarrage	millis();	temps = millis();
micros()	Obtenir le temps écoulé en microsecondes	micros();	temps = micros();
constrain()	Limiter une valeur entre deux bornes	constrain(valeur, min, max);	constrain(300, 0, 255);
map()	Changer une valeur d'échelle	map(valeur, min1, max1, min2, max2);	map(2048, 0, 4095, 0, 255);
min()	Retourner la plus petite valeur	min(valeur1, valeur2);	min(10, 20);
max()	Retourner la plus grande valeur	max(valeur1, valeur2);	max(10, 20);
abs()	Retourner la valeur absolue	abs(valeur);	abs(-25);
random()	Générer un nombre pseudo-aléatoire	random(min, max);	random(0, 100);
randomSeed()	Initialiser le générateur aléatoire	randomSeed(valeur);	randomSeed(analogRead(35));



L'API Arduino

API / fonction	Rôle	Syntaxe générique	Exemple
<code>attachInterrupt()</code>	Associer une fonction à une interruption	<code>attachInterrupt(broche, fonction, mode);</code>	<code>attachInterrupt(26, ISR_bouton, RISING);</code>
<code>detachInterrupt()</code>	Désactiver une interruption	<code>detachInterrupt(broche);</code>	<code>detachInterrupt(26);</code>
<code>Serial.begin()</code>	Initialiser la communication série	<code>Serial.begin(vitesse);</code>	<code>Serial.begin(115200);</code>
<code>Serial.print()</code>	Envoyer une donnée sur le port série	<code>Serial.print(donnee);</code>	<code>Serial.print(valeur);</code>
<code>Serial.println()</code>	Envoyer une donnée suivie d'un retour à la ligne	<code>Serial.println(donnee);</code>	<code>Serial.println("Bonjour");</code>
<code>Serial.available()</code>	Vérifier si des données sont disponibles	<code>Serial.available();</code>	<code>if (Serial.available() > 0)</code>
<code>Serial.read()</code>	Lire un caractère reçu sur le port série	<code>Serial.read();</code>	<code>char c = Serial.read();</code>

TD Blink

```
// faire clignoter la LED intégrée
```

```
// La fonction setup() est exécutée UNE SEULE FOIS  
// au démarrage ou après un RESET de la carte.
```

```
void setup() {
```

```
// LED BUILTIN correspond à la LED intégrée de l'Arduino UNO.  
// On doit configurer cette broche comme une _____.  
  pinMode(LED_BUILTIN, _____);}
```

```
// La fonction loop() est exécutée en boucle indéfiniment.
```

```
void loop() {
```

```
  // Allumer la LED :
```

```
  // HIGH place la sortie à l'état logique 1.  
  digitalWrite(LED_BUILTIN, _____);
```

```
  // Attendre 1 seconde.
```

```
  // delay() attend un nombre de millisecondes.  
  // 1000 ms = 1 seconde.
```

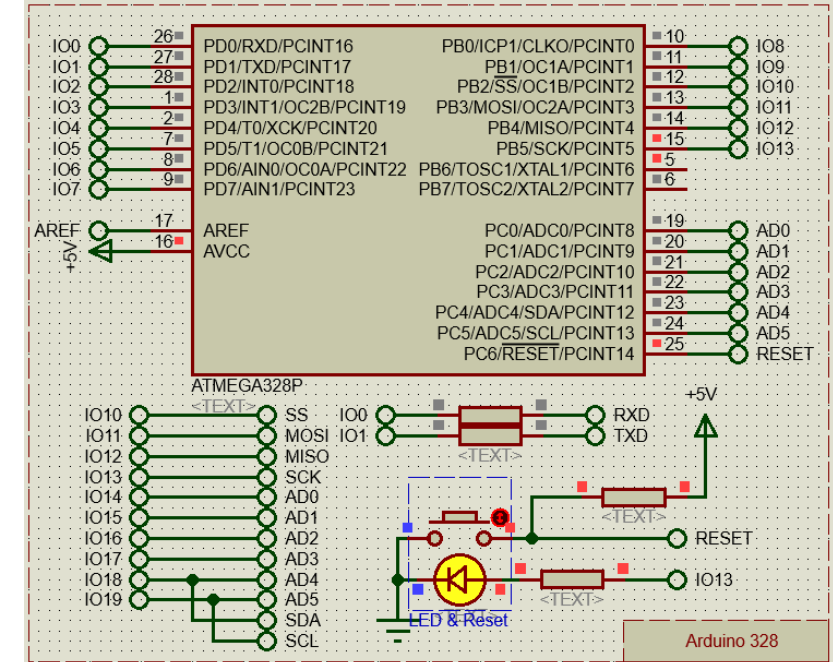
```
  delay(_____);
```

```
  // Éteindre la LED :
```

```
  // LOW place la sortie à l'état logique 0.  
  digitalWrite(LED_BUILTIN, _____);
```

```
  // Attendre à nouveau 1 seconde.
```

```
  delay(_____);}
```



Type, Variable, format

Exemples de déclarations :

```
int valeur;
float temperature;
bool etat;
const int LED = 8;
```

Structures de contrôle

```
if (...)
{
}
else
{
}

for (...)
{
}

while (...)
{
}
```

Différents types de données utilisables avec Arduino.

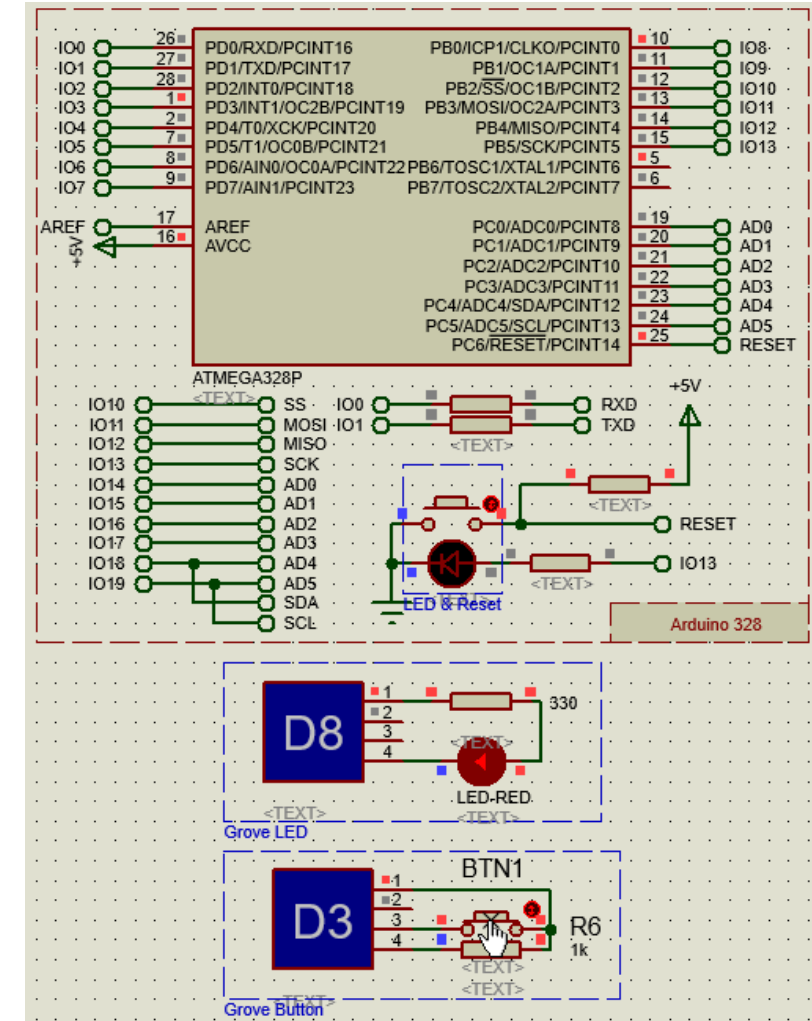
TYPE	Valeurs possibles	BITS	Octets
char	-128 à +127	8	1
int	-32768 à +32767	16	2
long	-2147483648 à +2147483647	32	4
Variables Décimales.			
float	$3,4 * 10^{(-38)}$ à $+3,4 * 10^{(38)}$	32	4
double	$1,7 * 10^{(-308)}$ à $+1,7 * 10^{(308)}$	64	8
La gestion des variables de type double dans Arduino est exactement la même que celle des variables de type float, sans gain de précision.			
Variables NON SIGNÉES : unsigned			
char	0 à 255	8	1
int	0 à 65535	16	2
long	0 à 4294967295	32	4
Variables propres au langage C d'Arduino			
byte	0 à +255	8	1
word	0 à +65535	16	2
boolean	0 ou 1	1	1
Variables booléennes			
boolean Nom Variable = FALSE ou TRUE			
Toute variable peut servir de variable booléenne : • Si elle vaut 0 ce sera interprété comme FALSE. • Si elle est $\neq 0$ ce sera interprété comme TRUE.			
Conversion Numérique analogique pour les entrées A0 à A5			
CAN	0 à 1023	10	2
"boolean" NomBrocheSortieBinaire = LOW ou HIGH			

TD bouton commande LED (TOR)

```

//Commande d'une LED par un bouton-poussoir
// Broche utilisée pour le bouton-poussoir
// Le bouton est connecté sur la broche numérique 3.
const int buttonPin = ____;
// Broche utilisée pour la LED
const int ledPin = ____;
// Variable permettant de mémoriser l'état du bouton
int buttonState = ____;
void setup() {
    // La LED est un composant de SORTIE.
    pinMode(ledPin, ____);
    // Le bouton est configuré en entrée avec
    // la résistance de rappel interne de l'Arduino.
    pinMode(buttonPin, ____);}
void loop() {
    // Lire l'état logique du bouton et
    // mémoriser le résultat dans buttonState.
    buttonState = ____ (buttonPin);
    // Tester l'état du bouton.
    // Avec INPUT PULLUP :
    // HIGH = bouton relâché
    // LOW = bouton appuyé
    if (buttonState == ____) {
        // Le bouton est relâché :
        // allumer la LED.
        digitalWrite(ledPin, ____);
    } else {
        // Le bouton est appuyé :
        // éteindre la LED.
        digitalWrite(ledPin, ____);}
}

```



Exercice : faire clignoter une LED 5 fois

```
// Broche de connexion de la LED
const int LED = _____;

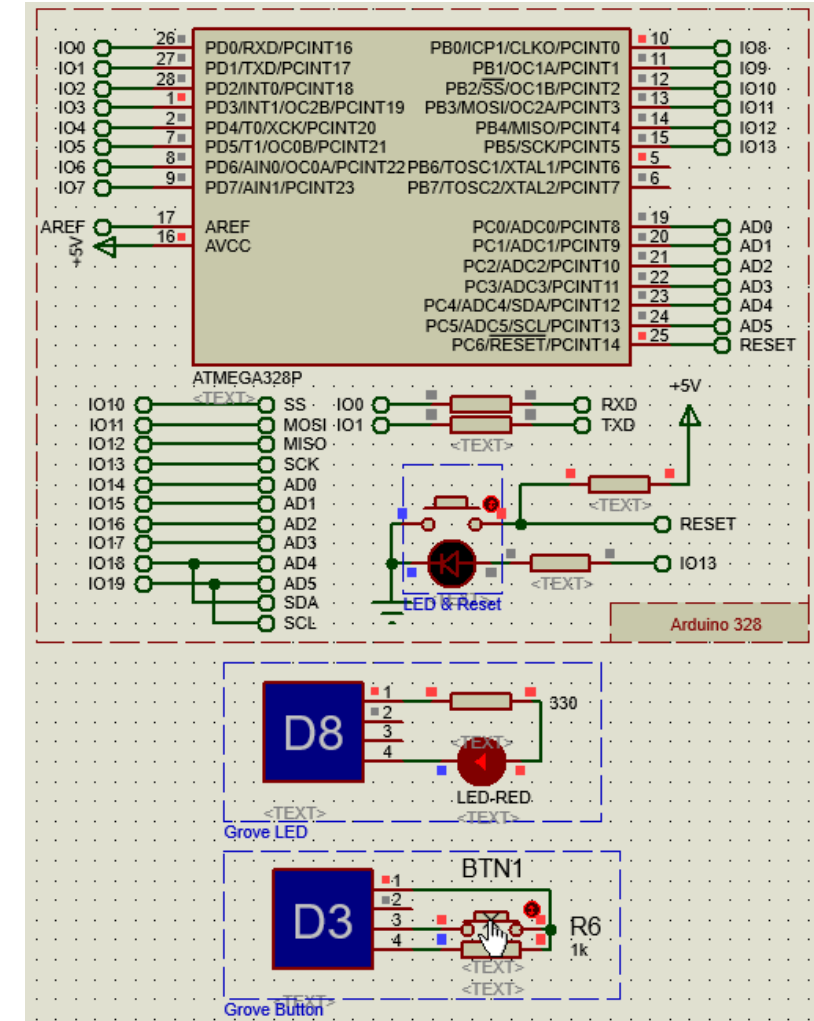
void setup()
{
    // Configuration de la broche de la LED
    pinMode(LED, OUTPUT);
}

void loop()
{
    // Répéter les instructions 5 fois
    for (int compteur = 0; compteur < _____;
        ++){
        // Allumer la LED
        digitalWrite(LED, _____);

        // Attendre 500 ms
        delay(_____);

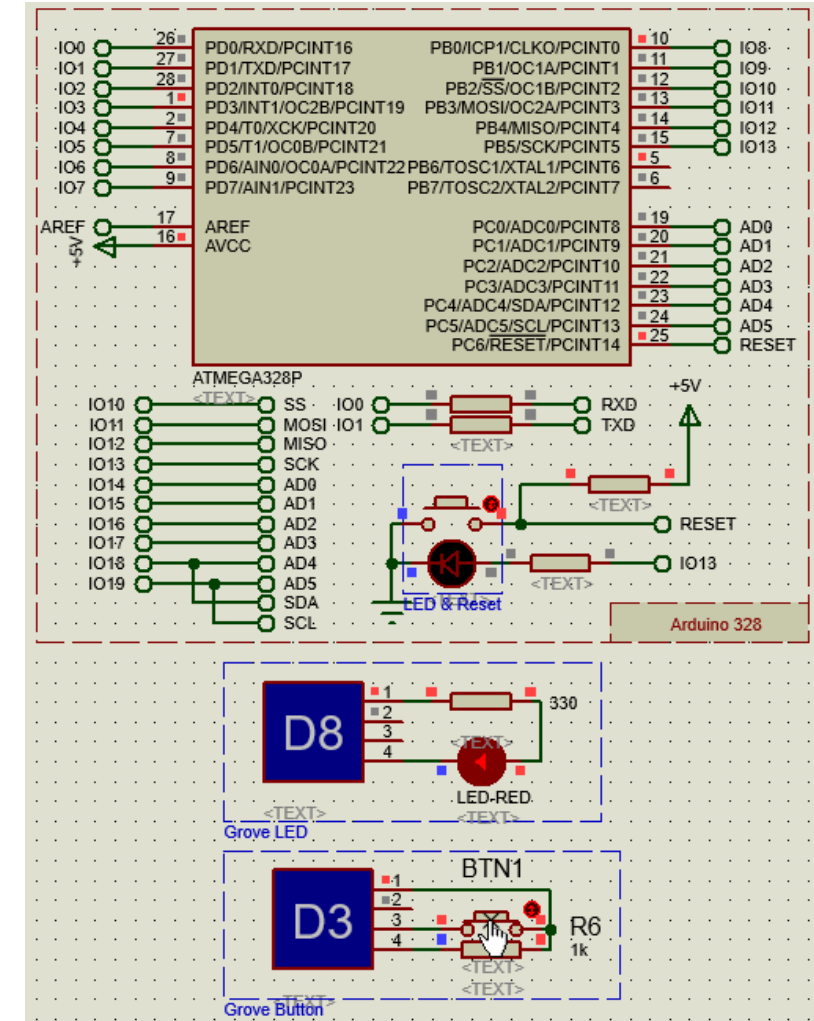
        // Éteindre la LED
        _____(LED, _____);

        // Attendre 500 ms
        _____(_____);
    }
}
```



Exercice : faire clignoter une LED 5 fois

```
// Broche de connexion de la LED
const int LED = 8;
void setup()
{
    // Configuration de la broche de la LED
    pinMode(LED, OUTPUT);
}
void loop()
{
    // Répéter les instructions 5 fois
    for (int compteur = 0; compteur < 5; compteur++)
    {
        // Allumer la LED
        digitalWrite(LED, HIGH);
        // Attendre 500 ms
        delay(500);
        // Éteindre la LED
        digitalWrite(LED, LOW);
        // Attendre 500 ms
        delay(500);
    }
}
```

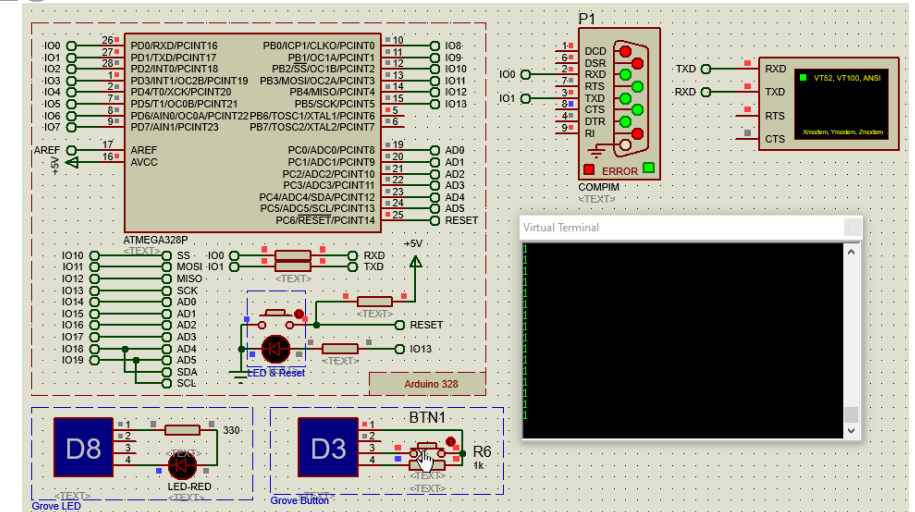




TD bouton commande LED (TOR)

```
// lecture d'un bouton-poussoir
// Affichage de l'état du bouton sur le moniteur série
// Broche numérique utilisée pour le bouton
int pushButton = ____;
void setup() {
    // Initialise la communication série à 9600 bauds
    Serial.begin(____);
    // Configure la broche du bouton comme une entrée
    pinMode(pushButton, ____);
}
void loop() {
    // Lit l'état logique de la broche du bouton
    // digitalRead() retourne HIGH ou LOW

    int buttonState = ____;
    // Affiche l'état du bouton sur le moniteur série
    Serial.__(buttonState);
    // Attendre 1 ms avant la prochaine lecture
    delay(____);
}
```

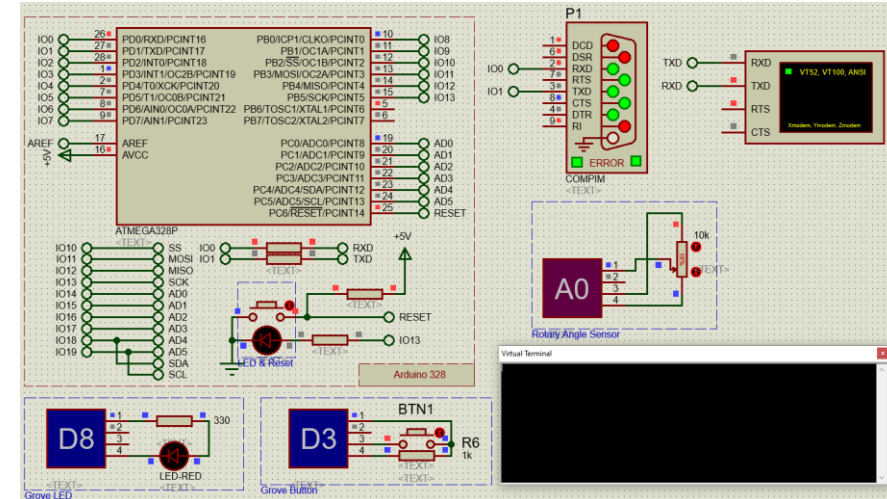


TD analogique Lire Potentiometre Ecrire PWM

```

// Potentiomètre → entrée analogique → vitesse de
// clignotement
// Broche analogique utilisée pour le potentiomètre
int sensorPin = ____;
// Broche numérique utilisée pour la LED
int ledPin = ____;
// Variable permettant de mémoriser la valeur du
// potentiomètre
int sensorValue = ____;
void setup() {
    // Configure la broche de la LED comme une sortie
    pinMode(ledPin, ____);
}
void loop() {
    // Lire la valeur analogique du potentiomètre
    // analogRead() retourne une valeur comprise entre 0
    // et 1023
    sensorValue = ____ (sensorPin);
    // Allumer la LED
    digitalWrite(ledPin, ____);
    // Attendre pendant la durée indiquée par le
    // potentiomètre
    // Plus sensorValue est grand, plus la LED reste
    // allumée longtemps
    delay(____);
    // Eteindre la LED
    digitalWrite(ledPin, ____);
    // Attendre la même durée
    delay(____);
}

```

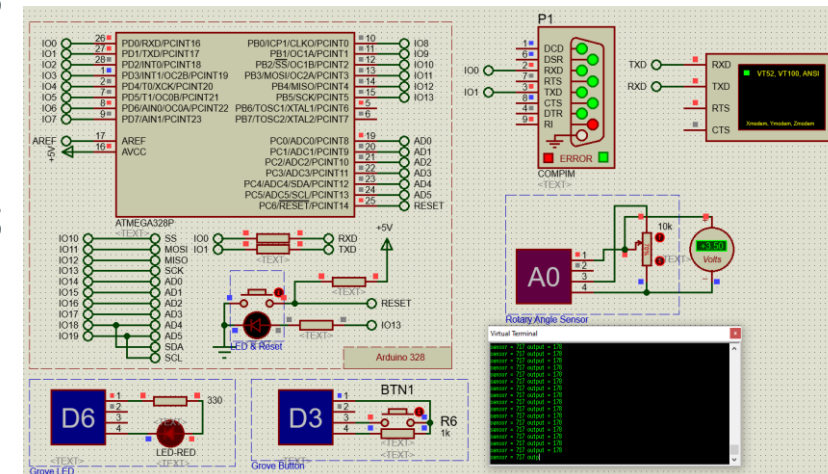


TD analogique Lire Potentiometre Ecrire PWM

```

// Potentiomètre → entrée analogique → sortie PWM
// Broche analogique du potentiomètre
const int analogInPin = ____;
// Broche PWM utilisée pour commander la LED
const int analogOutPin = ____;
// Valeur lue sur l'entrée analogique
int sensorValue = ____;
// Valeur convertie pour la sortie PWM
int outputValue = ____;
void setup() {
    // Initialiser la communication série à 9600 bauds
    Serial._____(____);
}
void loop() {
    // Lire la valeur du potentiomètre
    // Arduino UNO : valeur comprise entre 0 et 1023
    sensorValue = _____(analogInPin);
    // Convertir la valeur 0-1023 en valeur 0-255
    // afin de pouvoir commander la sortie PWM.
    outputValue = _____(sensorValue, 0, 1023, 0, 255);
    // Générer un signal PWM sur la sortie
    // outputValue détermine le rapport cyclique.
    analog_____ (analogOutPin, outputValue);
    // Afficher la valeur du potentiomètre
    Serial._____("sensor = ");
    Serial._____(sensorValue);
    // Afficher la valeur PWM
    Serial._____("\t output = ");
    Serial._____(outputValue);
    // Attendre 2 ms
    delay(____);
}

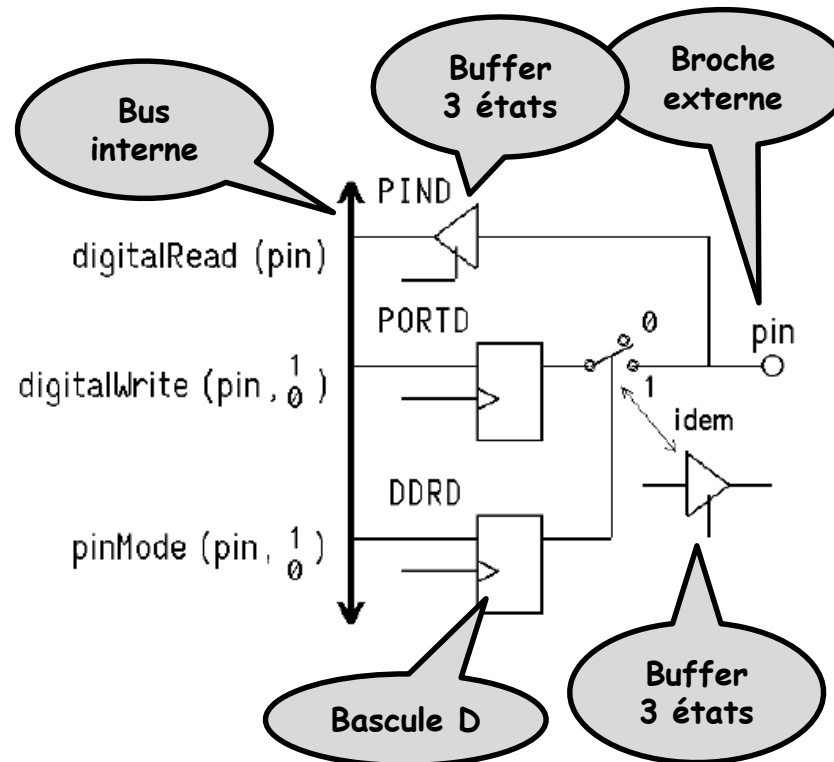
```





Pour mieux comprendre les mécanismes de lecture/écriture propres aux fonctions **pinMode()**, **digitalRead()** et **digitalWrite()** il faut étudier les structures matérielles auxquelles elles sont rattachées.

Voilà une représentation structurelle simplifiée d'une broche du port **D** et les correspondances avec les 3 instructions citées précédemment :



Simulation sous Proteus

L2 Informatique

UE S-E06-0207

